

EECS581: Project 01 Documentation

Group 6 - Team Members: Kaitlyn Bedgood, Marie Biernacki, Ximena Bustos, Sabelli Antebi Delmas, Greeshma Kunduri, Lauren Lee, Jaydine Stiles

Program Purpose: This program is a single-player Minesweeper puzzle game where players interact with a 10x10 grid. The objective is to safely uncover all non-mine cells by using numerical hints that indicate adjacent mines, flagging suspected mine locations, and avoiding detonation.

System Architecture Overview

System Components

File	main.py
Purpose	Displays the current state of the game to the viewer, processes user terminal inputs, and communicates with game_logic.py to ensure the minesweeper board is accurately updated.

Functions	
display_board	Renders the 10x10 grid with: - column labels A-J - row numbers 1-10 - appropriate cell symbols - # for covered - F for flagged - numbers 1-8 for adjacent mines - space for zero adjacent mines - * for revealed mines
main	Manages the primary game loop: - prompts user for mine count (10-20) - guarantees first click mine placement safety - processes user moves - U for uncover - F for flag

File	game_logic.py
Purpose	Enforces core game rules, manages mine placement triggers, processes player actions (uncovering and flagging), and tracks win/loss conditions.

Functions	
__init__	Initializes game logic instance, tracks the active board instance, remaining flag based on total mines, and current game state
uncover_cell	Evaluates a targeted cell's coordinates, prevents uncovering flagged or already uncovered cells, triggers a loss if a mine is hit, or uncovers standard numbered/empty cells.
recursive_reveal	Recursively uncovers neighboring cells when a selected cell has zero adjacent mines.
toggle_flag	Places or removes a flag on a covered cell while tracking and updating the total remaining available flags.
check_game_state	Scans the board to determine if the current game status is "Playing", "Game Over: Loss", or "Victory".

File	board_manager.py
Purpose	Manages the 10x10 layout grid as a two-dimensional collection of Cell objects, handles neighbor lookups, and controls initial random mine placement.

Functions	
__init__	Initializes the 10x10 2D array populated with individual Cell instances
get_cell	Retrieves and returns the specific Cell object located at a given row and column coordinates.
get_neighbors	Validates grid boundaries and returns a list of adjacent surrounding Cell objects for a given coordinate.

place_mines	Validates the user-specified mine count (10-20) and randomly distributes mines across the grid while guaranteeing the first selected coordinate remains completely safe and mine-free.
-------------	--

File	cell_real.py
Purpose	Represents an individual grid location on the 10x10 board, holding its specific gameplay properties and state data.

Functions	
__init__	Initializes the cell's starting attributes, including its visibility/flag state, whether it contains a mine, and the count of adjacent neighboring mines.

Data Flow

The steps below outline the lifecycle of a player's action. This process repeats continuously in a loop until game_logic.py determines if the player has won or lost.

1. Input: main.py

- The user enters a coordinate and action command via the command-line interface.
 - ex: "A 5 U"
- The input handling logic in main.py validates this input to ensure it falls within the required grid boundaries.
 - Columns: A-J
 - Rows: 1-10
 - Valid Actions: U for uncover, F for flag

2. Routing: main.py -> game_logic.py

- Once validated, the user input is routed to the game_logic module. Depending on the action chosen, main.py calls either the uncover_cell() or toggle_flag() methods within game_logic.py.

3. Update: game_logic.py -> board_manager.py -> cell.py

- game_logic.py evaluates the gameplay rules, such as checking if a mine was hit or if a recursive reveal is necessary.
- To apply the change, it queries board_manager.py to retrieve the desired Cell object.
- Then, it directly updates the cell's underlying data state.
 - 0 = covered
 - 1 = flagged
 - 2 = uncovered

iv. 3 = mine

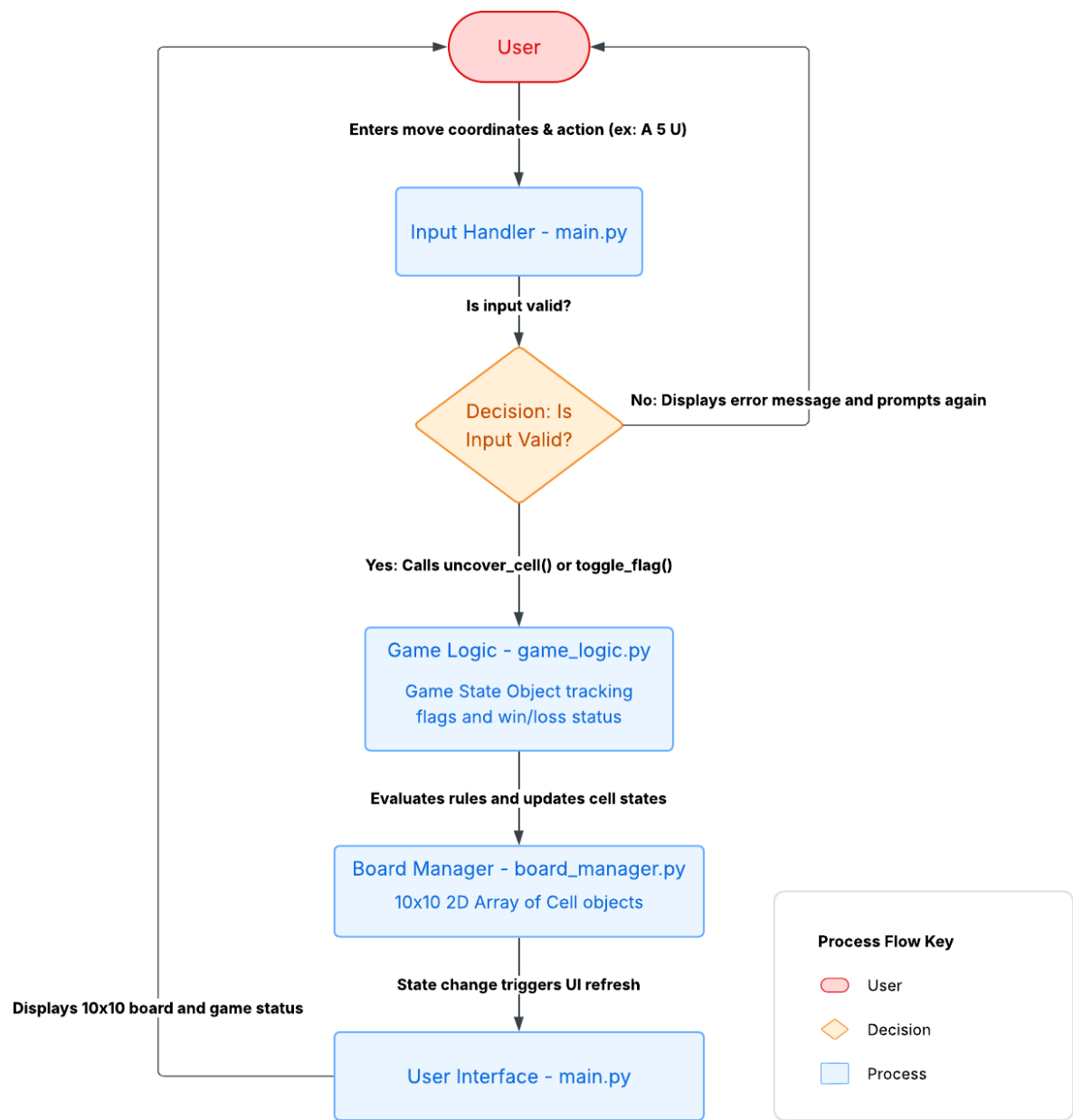
4. Display: main.py

- a. After the data structures are updated, the main game loop triggers the UI to refresh.
- b. The display_board function in main.py pulls the updated states from board_manager.py and renders the new 1-x1- grid and status indicators back to the user's terminal.

Key Data Structures Table

Data Structure	Python Files	Description
10x10 2D Array	board_manager.py cell_real.py	BoardManager module initializes a self.grid variable, which is a 2D list. This grid is populated by individual Cell objects. Each Cell object stores its specific state as an integer (0 = covered, 1 = flagged, 2 = uncovered number, 3 = mine), a boolean is_mine flag, and an adjacent_mines integer.
Game State Object	game_logic.py	The GameLogic instance acts as the primary tracker for the current session. It stores the overarching self.remaining_flags integer (total mines - placed flags) and the self.game_state string, which dynamically updates to exactly "Playing", "Game Over: Loss", or "Victory" based on player actions.

System Architecture: Process Flow Diagram



Person Hours Estimate Methodology

Our team implemented a Phase-Based Estimation methodology to objectively estimate person-hours. Tasks were categorized by complexity (Easy, Medium, and Hard) and the total estimated time E was calculated using the following formula:

timeU = time to understand requirements

timeC = time to code/work

timeT = time to test/review

$$E = \text{timeR} + \text{timeC} + \text{timeT}$$

To ensure accuracy, the coding phase was weighted higher for team members with beginner-level Python experience compared to those with advanced experience.

Task	Assigned	Complexity	Formula	Estimate (hours)
Code Template	Marie (Beginner)	Medium	timeU = 0.25 timeC = 1.5 timeT = 0.25	2
Github Tutorial Document	Kaitlyn	Easy	timeU = 0.25 timeC = 0.75 timeT = 0.5	1.5
Board Management	Kaitlyn	Easy	timeU = 0.25 timeC = 1 timeT = 0.25	1.5
Cell Object	Greeshma	Easy	timeU = 0.25 timeC = 0.5 timeT = 0.25	1
Mine Placement	Lauren	Hard	timeU = 0.5 timeC = 1 timeT = 0.5	2
Reveal Logic	Jaydine (Beginner)	Hard	timeU = 0.5 timeC = 1.5 timeT = 0.5	2.5
Flags + Win/Loss	Ximena	Hard	timeU = 0.5 timeC = 1 timeT = 0.5	2
Main Function	Sabelli	Easy	timeU = 0.25 timeC = 0.5	1

			timeT = 0.25	
Integration + Testing	Kaitlyn + Jaydine (Beginner)	Medium	timeU = 0.5 timeC = 1 timeT = 1	2.5
System Documentation + Architecture	Marie (Beginner)	Medium	timeU = 0.25 timeC = 1.75 timeT = 1	3
Git Version Control	Kaitlyn + Ximena	Medium	timeU = 0.25 timeC = 1 timeT = 1	2.25

Actual Accounting of Person-Hours

Kaitlyn Bedgood

Day	Task	Hours Worked
9/8	Team Meeting w/ TA creation of board	1 0.5
9/12	Board Development + GitHub Reorganizing Virtual Team Meeting	2.25 0.75
9/14	GitHub Management	0.75
9/15	Team Meeting w/ TA	0.75
9/16	board_manager.py integration, removal of board.py	1
9/17	project testing, integration, removal of cell.py Virtual Team Meeting	2 0.25
9/20	Final Testing + Submission	1

Marie Biernacki

Day	Task	Hours Worked
9/8	Team Meeting w/ TA	1
9/12	Virtual Team Meeting Code Templates	0.75 2
9/13	System Documentation	0.25
9/15	Team Meeting w/ TA	0.75
9/16	System Documentation	2
9/17	System Documentation Virtual Team Meeting	0.5 0.25
9/19	Final Documentation Review	0.75

Ximena Bustos

Day	Task	Hours Worked
9/8	Team Meeting w/ TA	1
9/12	Virtual Team Meeting	0.75
9/14	Wrote for game_logic.py	3
9/15	Team Meeting w/ TA	0.75
9/17	Virtual Team Meeting Added comments to my code	0.25 1

Sabelli Antebi Delmas

Day	Task	Hours Worked
9/8	Team Meeting w/ TA	1
9/13	UI Completed	1.5
9/15	Team Meeting w/ TA	0.75
9/17	Virtual Team Meeting	0.25

Greeshma Kunduri

Day	Task	Hours Worked
9/8	Team Meeting w/ TA	1
9/12	Virtual Team Meeting	0.5
9/15	Team Meeting w/ TA	0.75
9/17	cell_real.py + comments	0.5

Lauren Lee

Day	Task	Hours Worked
9/8	Team Meeting w/ TA	1
9/14	Adding github repo + code	1
9/15	Team Meeting w/ TA Coding + Updating	0.75 1
9/17	Virtual Team Meeting	0.25

Jaydine Stiles

Day	Task	Hours Worked
9/8	Team Meeting w/ TA	1
9/12	Virtual Team Meeting	0.75
9/15	Team Meeting w/ TA Writing Reveal Logic	0.75 2
9/17	Testing Virtual Team Meeting	1.25 0.25